

现代编程思想

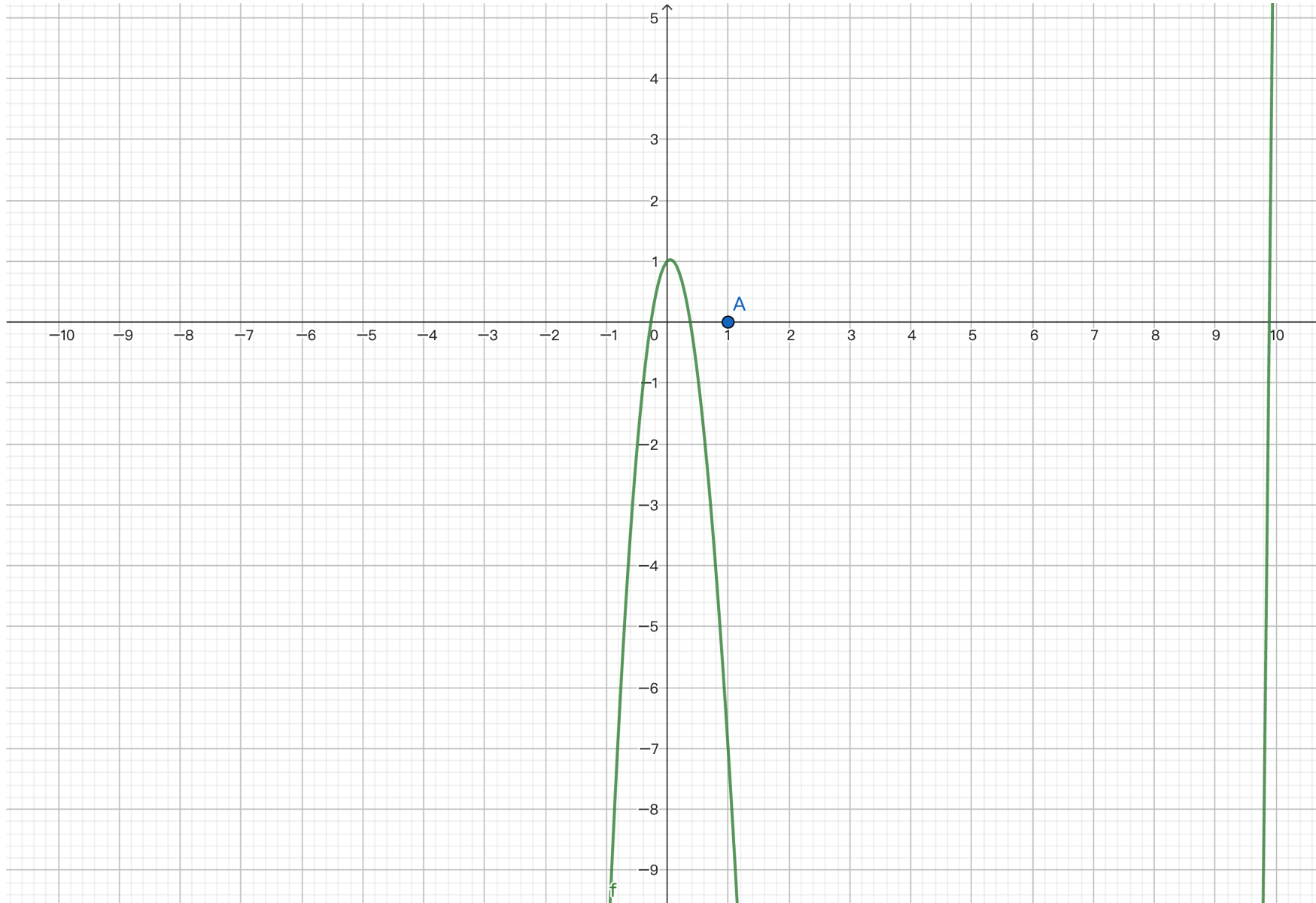
案例：自动微分

月兔公开课课程组

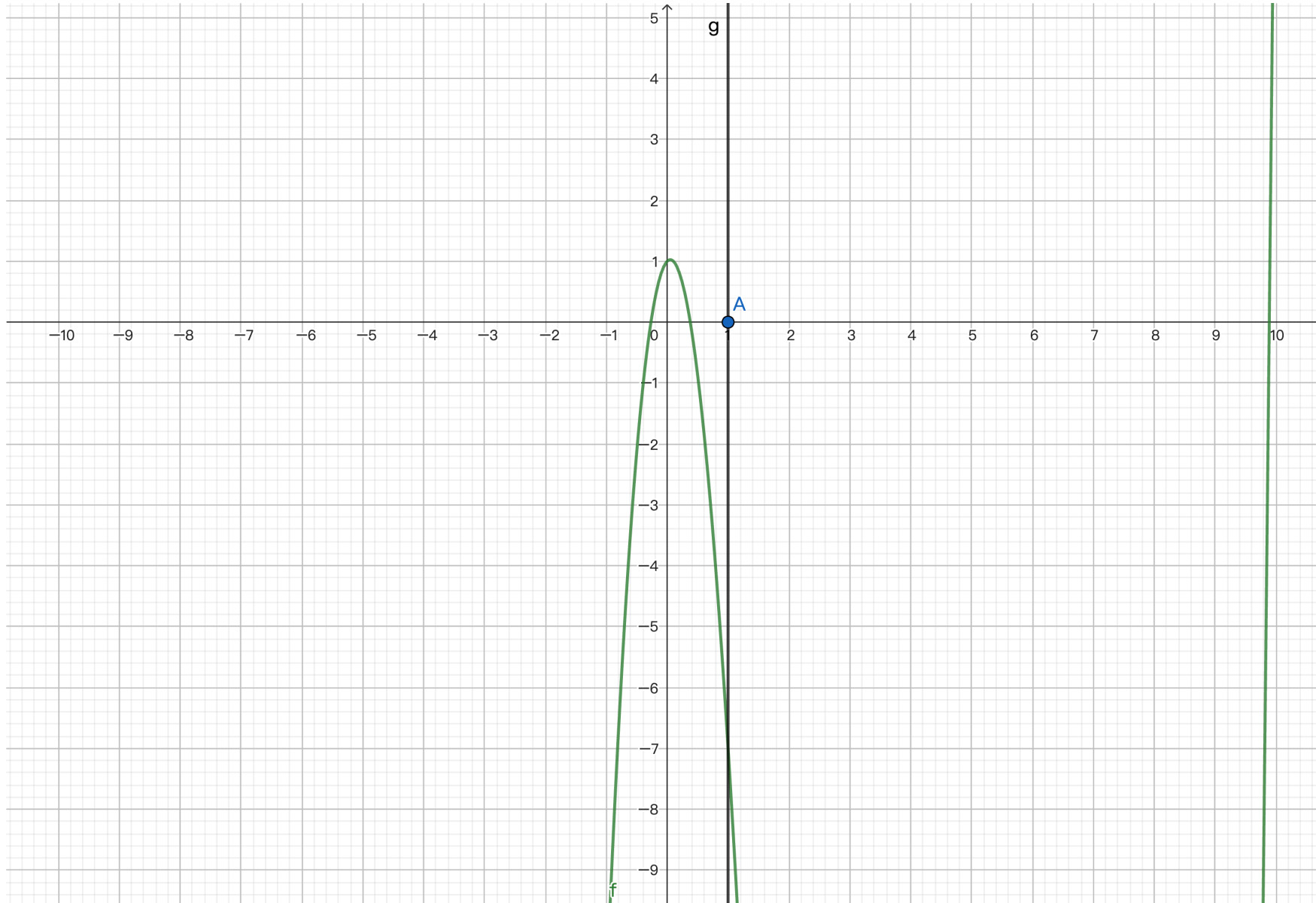
微分

- 微分被应用于机器学习领域
 - 利用梯度下降求局部极值
 - 牛顿迭代法求函数解: $x^3 - 10x^2 + x + 1 = 0$
- 我们今天研究简单的函数组合
 - 例: $f(x_0, x_1) = 5x_0^2 + x_1$
 - $f(10, 100) = 600$
 - $\frac{\partial f}{\partial x_0}(10, 100) = 100$
 - $\frac{\partial f}{\partial x_1}(10, 100) = 1$

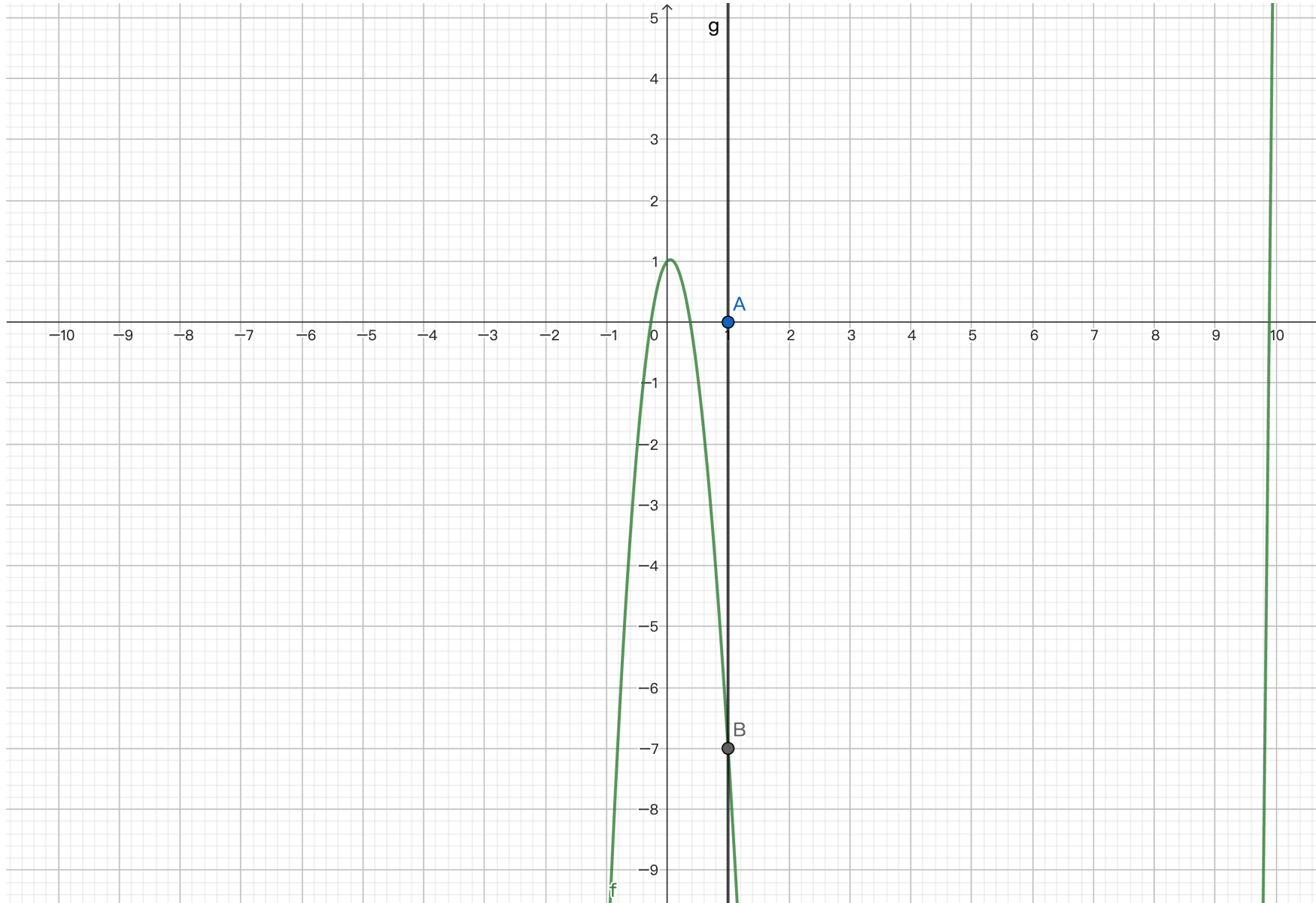
牛顿迭代法



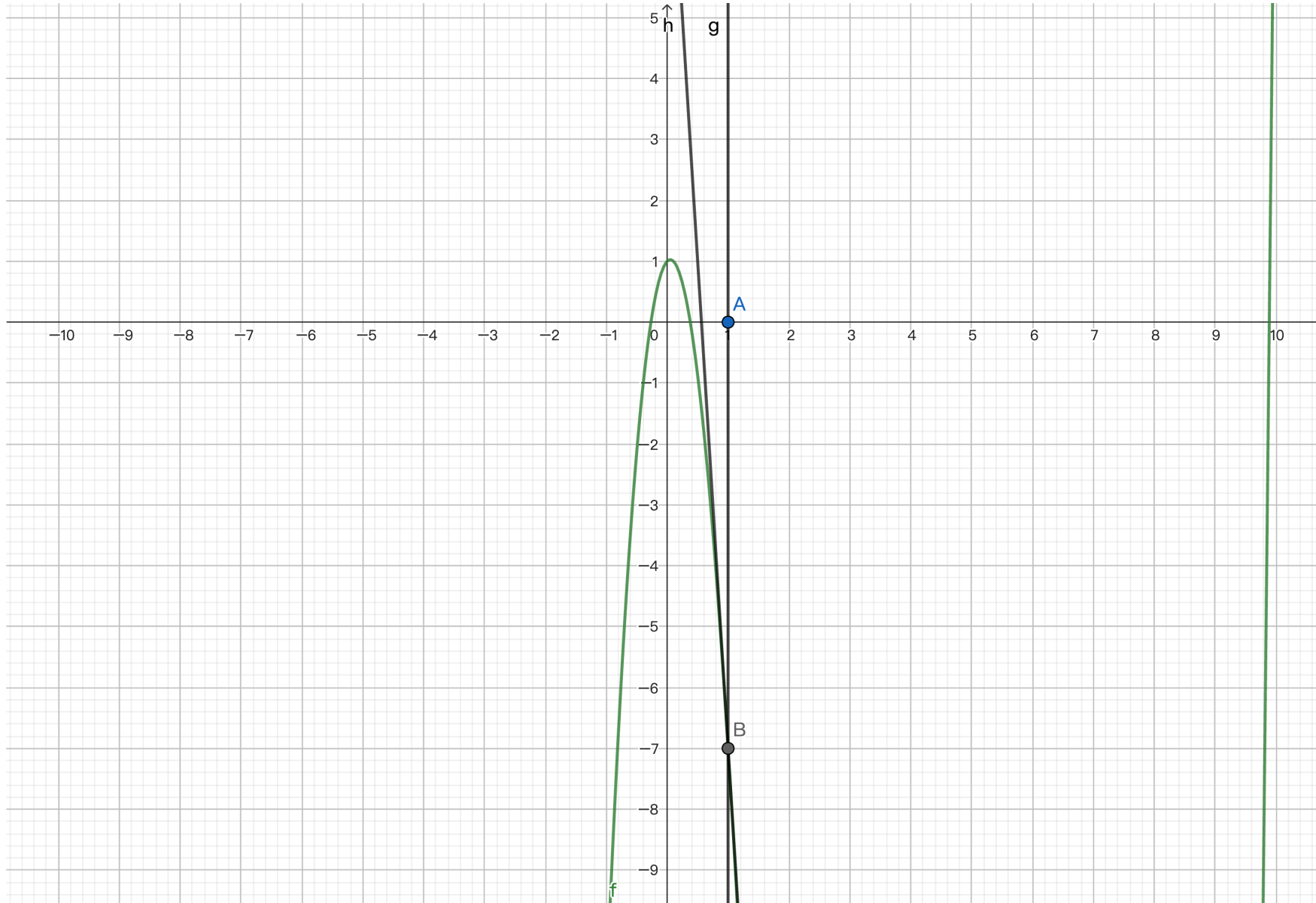
牛顿迭代法



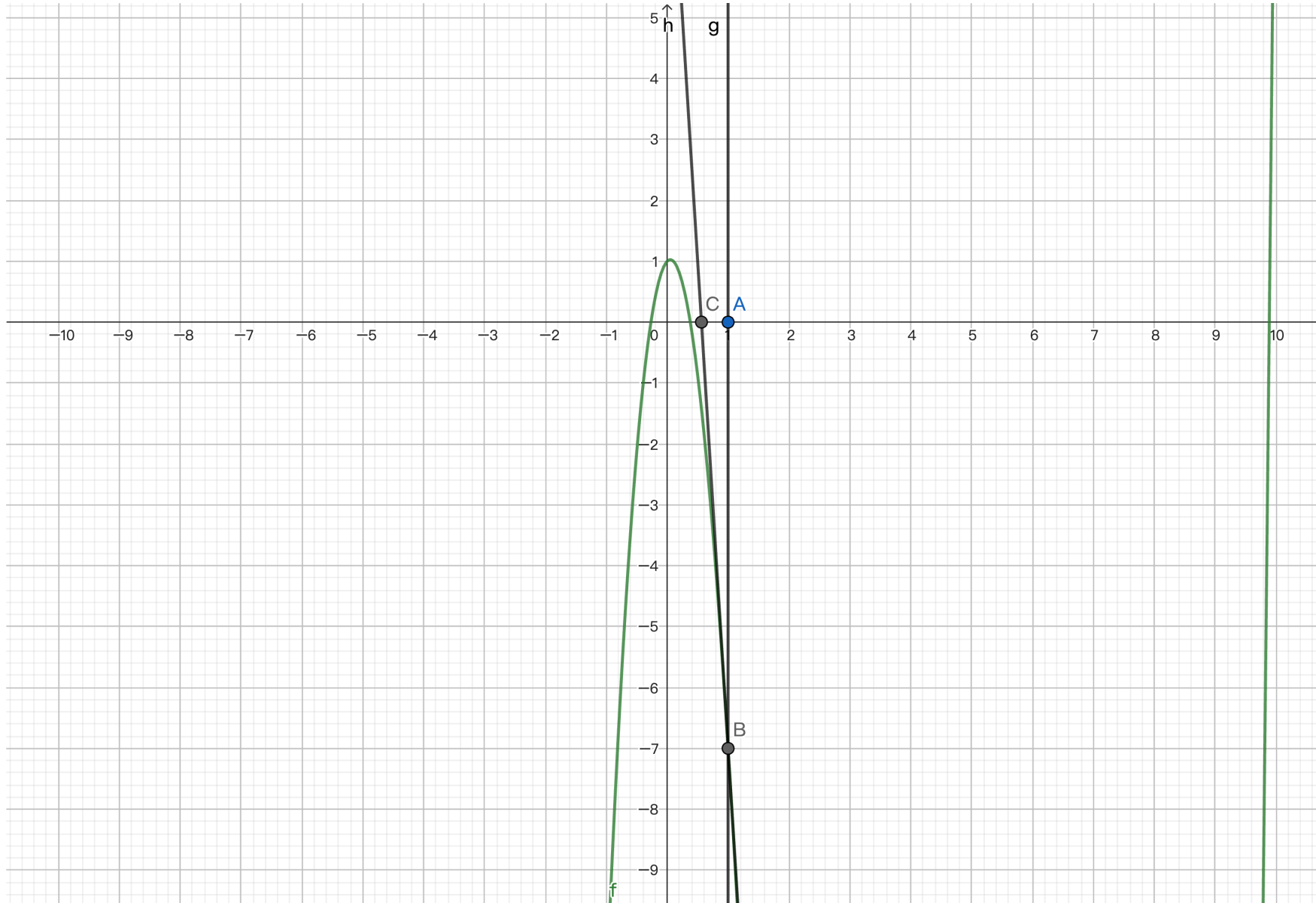
牛顿迭代法



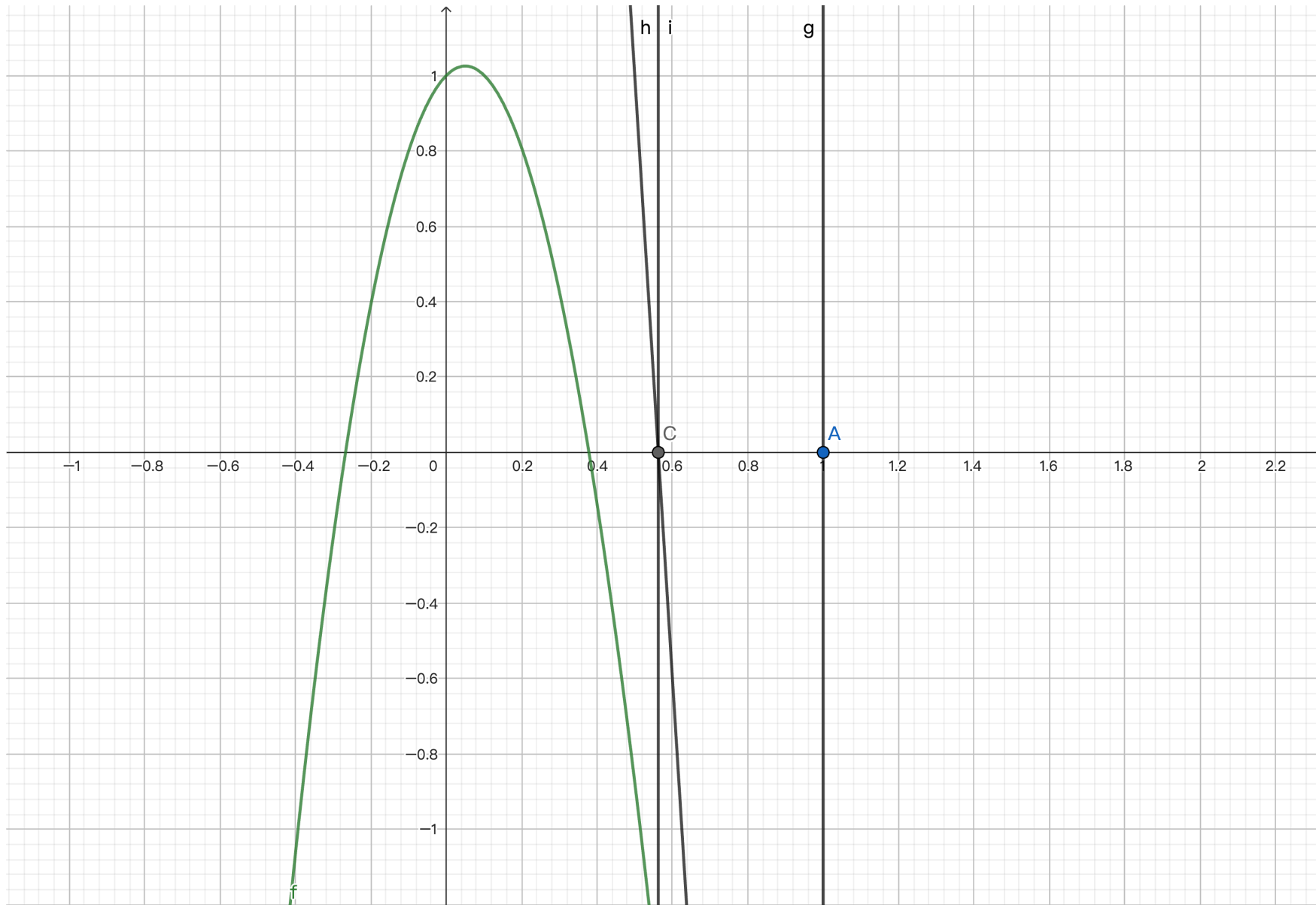
牛顿迭代法



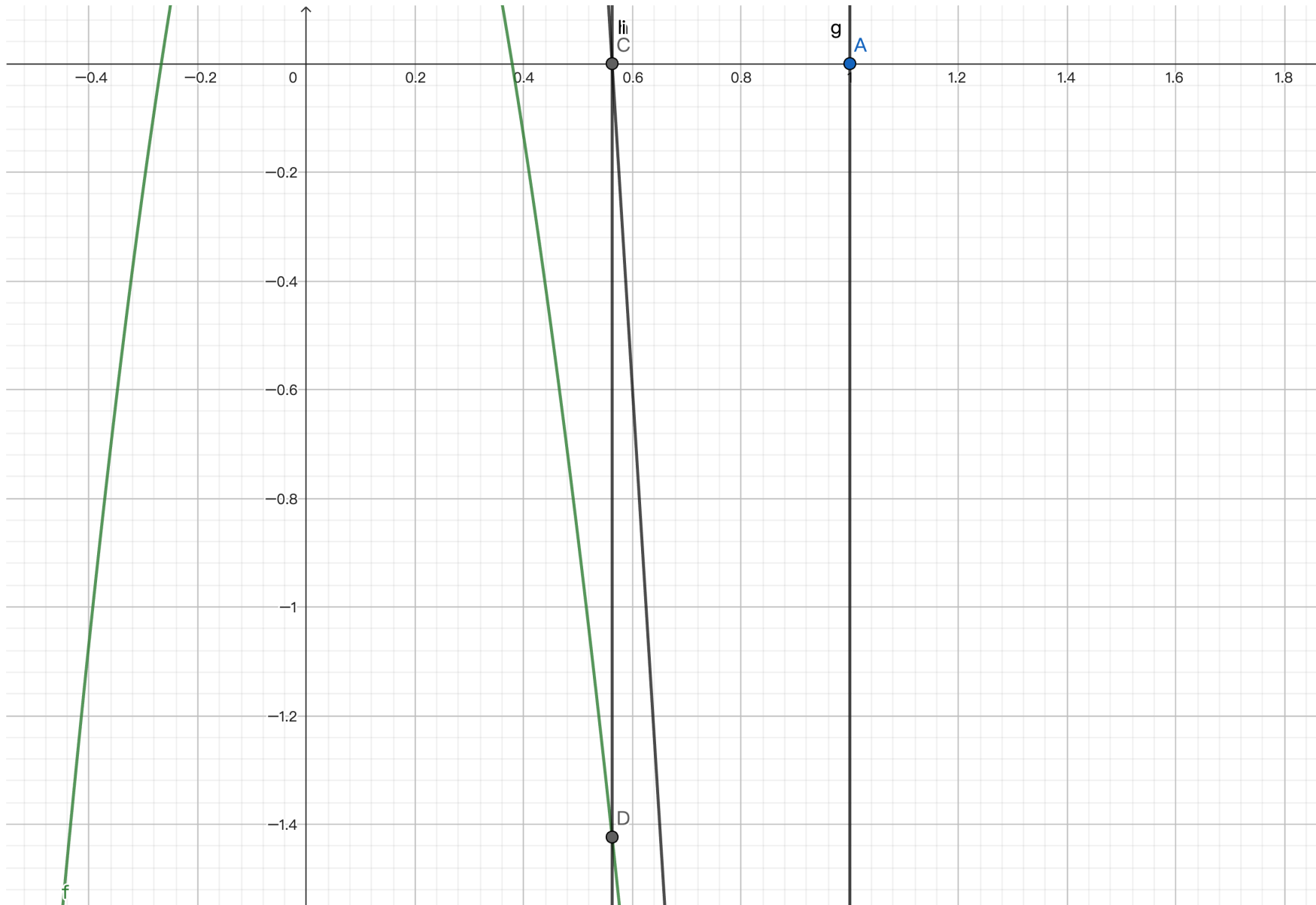
牛顿迭代法



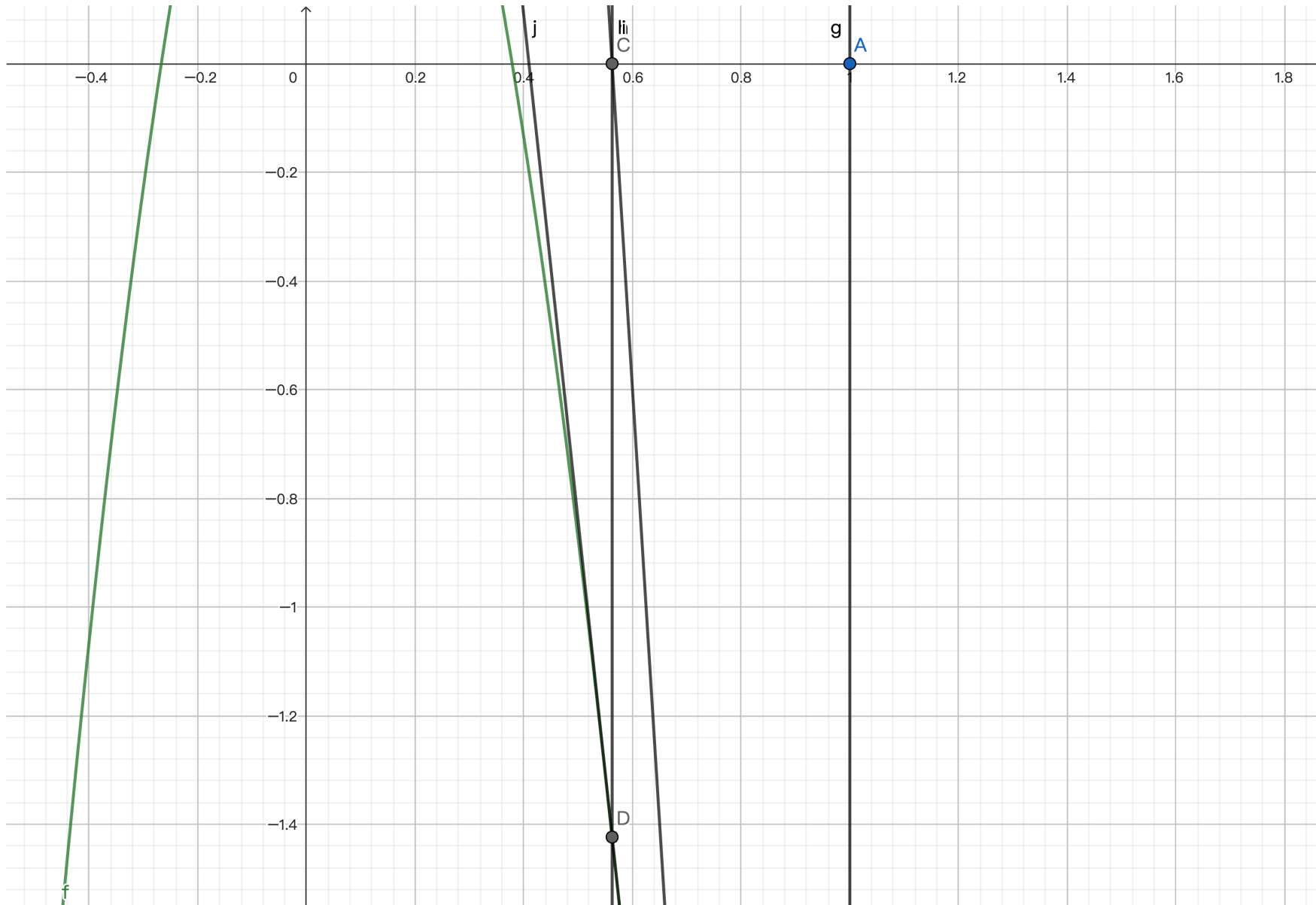
牛顿迭代法



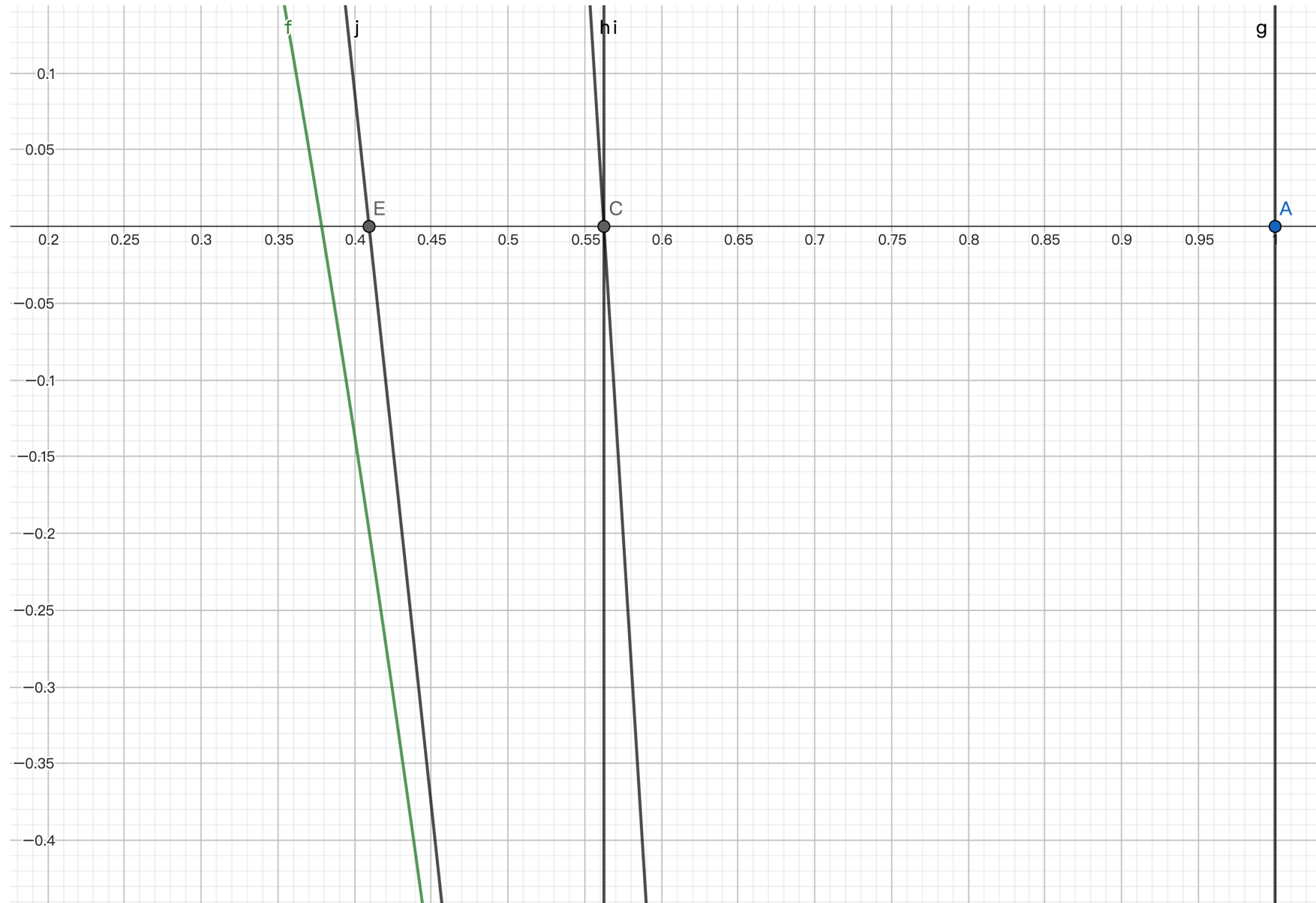
牛顿迭代法



牛顿迭代法



牛顿迭代法



- 函数微分的几种方式

- 手动微分：纯天然计算器

- 缺点：对于复杂表达式容易出错

- 数值微分： $\frac{f(x+\delta x) - f(x)}{\delta x}$

- 缺点：计算机无法精准表达小数，且绝对值越大，越不精准

- 符号微分： `Mul(Const(2), Var(0)) -> Const(2)`

- 缺点：计算结果可能复杂；可能重复计算；难以直接利用语言原生控制流
 - 例如无法简单地写出这样的函数：

```
1. fn[N : Number] max(x : N, y : N) -> N { if x < y { y } else { x } }
```

- 自动微分：利用复合函数求导法则、由基本运算组合进行微分

- 分为前向微分和后向微分

- 我们以符号微分定义表达式构建的一种语义

```
1. enum Symbol {
2.     Constant(Double)
3.     Var(Int) // x0, x1, x2, ...
4.     Add(Symbol, Symbol)
5.     Mul(Symbol, Symbol)
6. } derive(Show)
7.
8. // 定义简单构造器, 并重载运算符
9. fn Symbol::constant(d : Double) -> Symbol { Constant(d) }
10. fn Symbol::variable(i : Int) -> Symbol { Var(i) }
11. impl Add for Symbol with add(f1, f2) { Add(f1, f2) }
12. impl Mul for Symbol with mul(f1, f2) { Mul(f1, f2) }
13.
14. // 计算函数值
15. fn Symbol::compute(f : Symbol, input : Array[Double]) -> Double {
16.     ...
17. }
```

- 利用函数求导法则，我们计算函数的（偏）导数

- $\frac{\partial f}{\partial x_i} = 0$ 如果 f 为常值函数
- $\frac{\partial x_i}{\partial x_i} = 1, \frac{\partial x_j}{\partial x_i} = 0, i \neq j$
- $\frac{\partial (f+g)}{\partial x_i} = \frac{\partial f}{\partial x_i} + \frac{\partial g}{\partial x_i}$
- $\frac{\partial (f \times g)}{\partial x_i} = \frac{\partial f}{\partial x_i} \times g + f \times \frac{\partial g}{\partial x_i}$

- 月兔实现：

```
1. fn Symbol::differentiate(self : Symbol, val : Int) -> Symbol {
2.   match self {
3.     Constant(_) => Constant(0.0)
4.     Var(i) => if i == val { Constant(1.0) } else { Constant(0.0) }
5.     Add(f1, f2) => f1.differentiate(val) + f2.differentiate(val)
6.     Mul(f1, f2) => f1 * f2.differentiate(val) + f1.differentiate(val) * f2
7.   }
8. }
```

符号微分

- 例子: $y = 5x_0^2 + x_1$, $\frac{\partial y}{\partial x_0} = 10x_0$, $\frac{\partial y}{\partial x_0}(10, 100) = 100$

```

1. fn example() -> Symbol {
2.   Constant(5.0) * Variable(0) * Variable(0) + Variable(1)
3. }
4. test {
5.   let func = example() // 函数的抽象语法树
6.   let diff_0_func = func.differentiate(0) // 对x_0的偏微分
7.   inspect(diff_0_func.compute([10.0, 100.0]), content="100")
8. }

```

- 需要化简: $\frac{\partial y}{\partial x_0} = ((0 \times x_0 + 5 \times 1) \times x_0 + (5 \times x_0) \times 1) + 0$

符号微分

- 微分的结果: $((0 \times x_0 + 5 \times 1) \times x_0 + (5 \times x_0) \times 1) + 0$
- 我们可以在构造期间进行化简

```
1. impl Add for Symbol with add(f1 : Symbol, f2 : Symbol) -> Symbol {
2.     match (f1, f2) {
3.         (Constant(0.0), a) => a // 0 + a = a
4.         (Constant(a), Constant(b)) => Constant(a + b) // 常数直接计算
5.         (a, Constant(_) as c) => c + a
6.         (Mul(n, Var(x1)), Mul(m, Var(x2))) if x1 == x2 =>
7.             Mul(m + n, Var(x1)) // n * x + m * x = (m + n) * x
8.         _ => Add(f1, f2)
9.     }
10. }
```


符号微分

- 我们可以在构造期间进行化简

```
1. impl Mul for Symbol with mul(f1 : Symbol, f2 : Symbol) -> Symbol {
2.     match (f1, f2) {
3.         (Constant(0.0), _) => Constant(0.0) // 0 * a = 0
4.         (Constant(1.0), a) => a              // 1 * a = 1
5.         (Constant(a), Constant(b)) => Constant(a * b)
6.         (a, Constant(_) as c) => c * a
7.         _ => Mul(f1, f2)
8.     }
9. }
```

- 化简效果

$$((0 \times x_0 + 5 \times 1) \times x_0 + (5 \times x_0) \times 1) + 0 \equiv 10 \times x_0$$

自动微分

- 链式法则：如果 $y = f(g(x))$ ，那么 $\frac{\partial y}{\partial x}(x_0) = \frac{\partial f}{\partial g}(g(x_0)) \times \frac{\partial g}{\partial x}(x_0)$
- 如果 $y = t_1(t_2(\dots(t_k(x))))$ ，那么 $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial t_1} \frac{\partial t_1}{\partial t_2} \dots \frac{\partial t_{k-1}}{\partial t_k} \frac{\partial t_k}{\partial x}$
- 多变量链式法则：如果 $y = f(t_1, \dots, t_n)$ ，其中每个 t_i 都是 x 的函数，那么 $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial t_1} \frac{\partial t_1}{\partial x} + \dots + \frac{\partial y}{\partial t_n} \frac{\partial t_n}{\partial x}$
- 例如， $y = x_0^2 \times x_1$ ，

$$\frac{\partial y}{\partial x_0} = \frac{\partial y}{\partial x_0^2} \times \frac{\partial x_0^2}{\partial x_0} + \frac{\partial y}{\partial x_1} \frac{\partial x_1}{\partial x_0} = x_1 \times 2x_0 + x_0^2 \times 0 = 2x_0x_1$$

前向微分

- 利用求导法则直接计算微分，同时计算 $f(a)$ 与 $\frac{\partial f}{\partial x_i}(a)$
 - 简单理解：计算 $(f \times g)' = f' \times g + f \times g'$ 需要同时计算 f 与 f'
- 对偶数 (Dual Numbers)

```
1. struct Dual {  
2.     value : Double      // 当前节点值    f  
3.     derivative : Double // 当前节点导数  f'  
4. }  
5. fn Dual::constant(d : Double) -> Dual {  
6.     { value: d, derivative: 0.0 }  
7. }  
8. fn Dual::new(d : Double, diff : Bool) -> Dual {  
9.     { value: d, derivative: if diff { 1.0 } else { 0.0 } }  
10. }
```

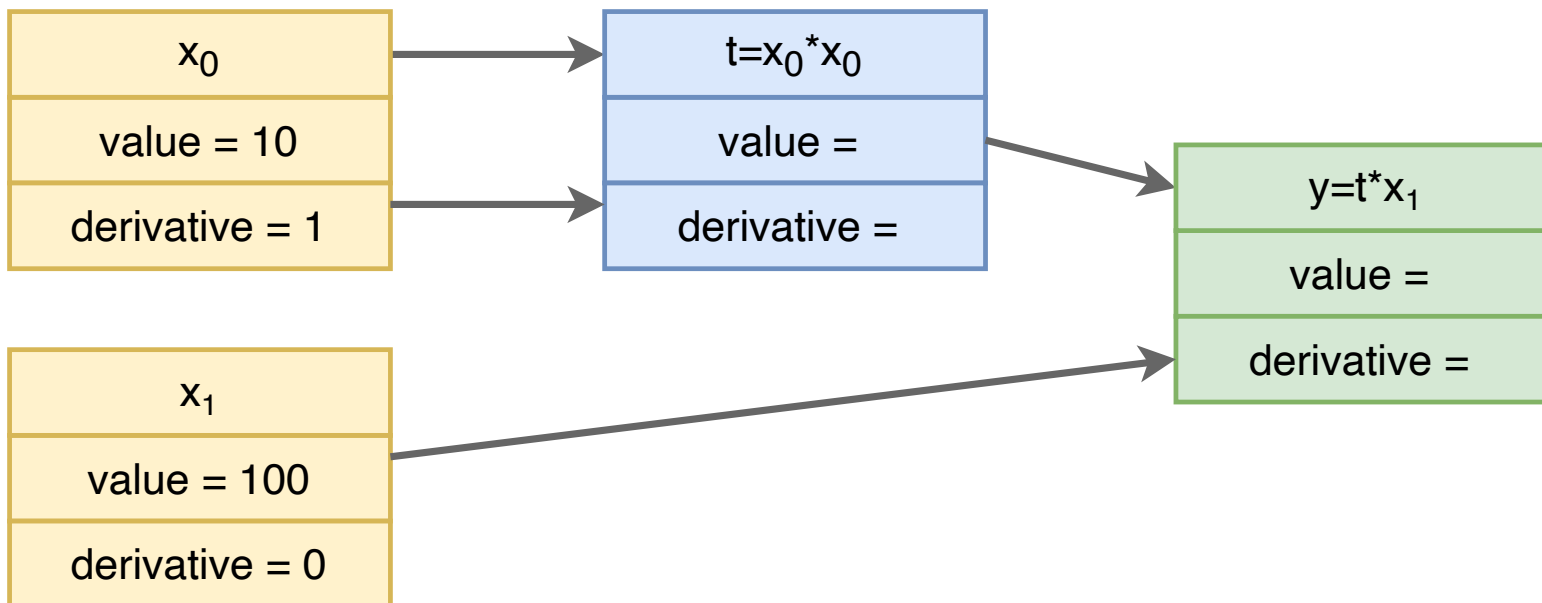
前向微分

- 原理: $(f + g)' = f' + g'$, $(f \times g)' = f' \times g + f \times g'$
- 通过加法和乘法构造新的对偶数

```
1. impl Add for Dual with add(f : Dual, g : Dual) -> Dual {
2.     {
3.         value: f.value + g.value,
4.         derivative: f.derivative + g.derivative, // f' + g'
5.     }
6. }
7.
8. impl Mul for Dual with mul(f : Dual, g : Dual) -> Dual {
9.     {
10.        value: f.value * g.value,
11.        derivative: f.value * g.derivative + g.value * f.derivative,
12.        // f * g' + g * f'
13.    }
14. }
```

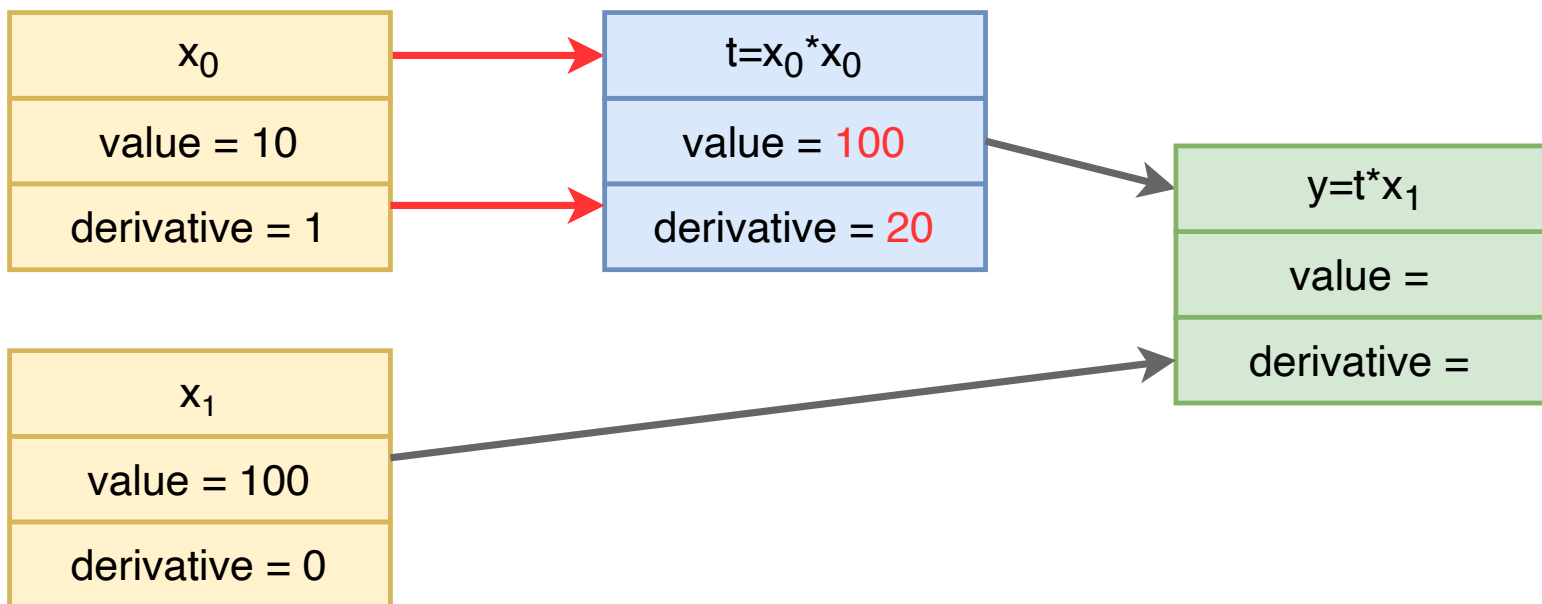
前向微分

- 例: $f(x_0, x_1) = x_0^2 \times x_1$, 求 $\frac{\partial f}{\partial x_0}$ ($x_0 = 10, x_1 = 100$)



前向微分

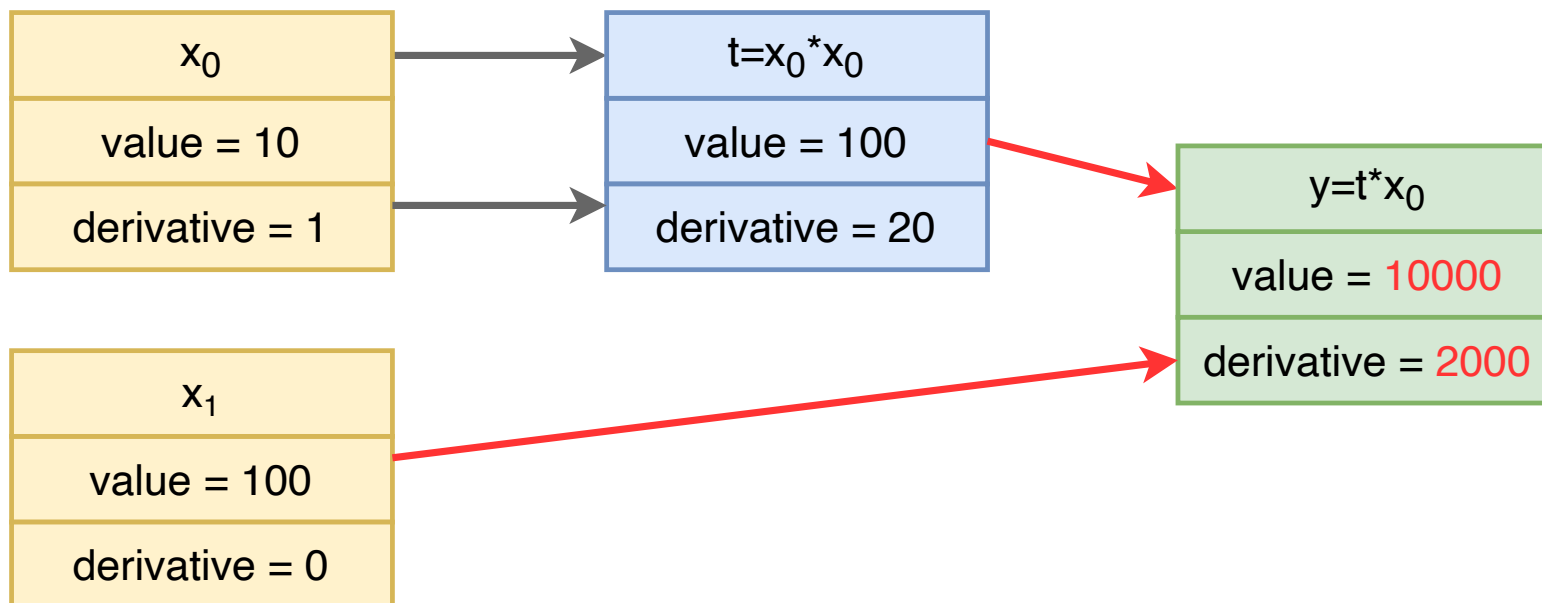
- 例: $f(x_0, x_1) = x_0^2 \times x_1$, 求 $\frac{\partial f}{\partial x_0}$ ($x_0 = 10, x_1 = 100$)
- 第一步, 令 $t = x_0^2$



- 求出 $t = 100, \frac{\partial t}{\partial x_0} = 20$

前向微分

- 例: $f(x_0, x_1) = x_0^2 \times x_1$, 求 $\frac{\partial f}{\partial x_0}$ ($x_0 = 10, x_1 = 100$)
- 第二步, 令 $y = t \times x_1$



- 求出 $y = 10000, \frac{\partial y}{\partial x_0} = 2000$

前向微分案例：牛顿迭代法求零点

- $f = x^3 - 10x^2 + x + 1$

```
1. fn example_newton(x : Dual) -> Dual {  
2.   x * x * x + Dual::constant(-10.0) * x * x + x + Dual::constant(1.0)  
3. }
```


前向微分案例：牛顿迭代法求零点

- 通过循环进行迭代

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

```
1. test {
2.   let mut x = 1.0 // 迭代起点
3.   while true {
4.     let { value, derivative } = example_newton(Variable(x, true))
5.     if (value / derivative).abs() < 1.0e-9 {
6.       break // 精度足够，终止循环
7.     }
8.     x -= value / derivative
9.   }
10.  inspect(x, content="0.37851665401644224")
11.  inspect(example_newton(x), content="-4.604427950027912e-11")
12. }
```

后向微分

- 前向微分：
 - 从输入开始，将微分结果逐步向前传递到每个输出
 - 对于每个输入 x_i 求导都要重新完整地计算一遍
 - 适用于输入少，输出多
- 后向微分：
 - 从输出开始，将微分结果逐步向后传递给每个输入
 - 一次计算出 y 对于所有 x_i 的导数
 - 适用于输入多，输出少

- 原理回顾：多变量链式法则 $\frac{\partial y}{\partial x} = \frac{\partial y}{\partial t_1} \frac{\partial t_1}{\partial x} + \dots + \frac{\partial y}{\partial t_n} \frac{\partial t_n}{\partial x}$
- 使用 `Node` 表示计算节点； `Edge` 表示节点之间的连接
- 需前向计算值，再后向传播导数

```
1. struct Node {  
2.     v : Double           // 节点的值 v  
3.     mut grad : Double    // 输出对于该节点的导数 dy/dv  
4.     parents : Array[Edge] // 指向父节点的边  
5. }  
6.  
7. type Var = Int  
8. struct Edge {  
9.     parent : Var          // 父节点索引  
10.    w : Double            // 权重  
11. }
```

后向微分

- 使用 `Tape` 记录节点计算顺序

```
1. struct Tape {  
2.   nodes : Array[Node]  
3. } derive(Show)  
4.  
5. fn Tape::new() -> Tape {  
6.   { nodes: [] }  
7. }  
8.  
9. fn Tape::push(self : Tape, v : Double, parents : Array[Edge]) -> Var {  
10.   let id = self.nodes.length()  
11.   self.nodes.push({ v, grad: 0.0, parents })  
12.   id  
13. }  
14.  
15. fn Tape::input(self : Tape, v : Double) -> Var { self.push(v, []) }
```

后向微分

- 使用 `Tape` 计算加法和乘法

```
1. fn Tape::add(self : Tape, a : Var, b : Var) -> Var {
2.   let va = self.val(a)
3.   let vb = self.val(b)
4.   // d(a+b)/da = 1, d(a+b)/db = 1
5.   self.push(va + vb, [{ parent: a, w: 1.0 }, { parent: b, w: 1.0 }])
6. }
7.
8. fn Tape::mul(self : Tape, a : Var, b : Var) -> Var {
9.   let va = self.val(a)
10.  let vb = self.val(b)
11.  // d(a*b)/da = b, d(a*b)/db = a
12.  self.push(va * vb, [{ parent: a, w: vb }, { parent: b, w: va }])
13. }
```

后向微分

- 从后向前计算导数（梯度）

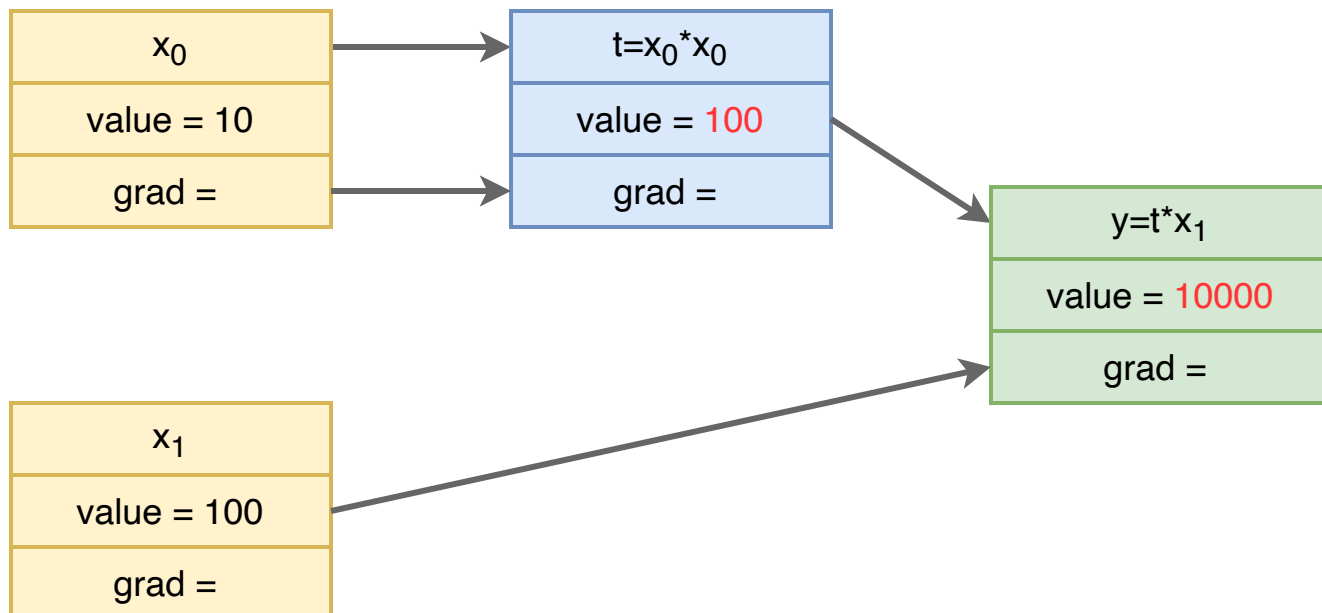
```
1. fn Tape::backward(self : Tape, out : Var) -> Unit {
2.     self.nodes[out].grad = 1.0
3.     // 逆序遍历：把每个节点的导数分发给它的父节点
4.     for i = self.nodes.length() - 1; i >= 0; i = i - 1 {
5.         let g = self.nodes[i].grad
6.         for e in self.nodes[i].parents {
7.             self.nodes[e.parent].grad += g * e.w
8.         }
9.     }
10. }
```

后向微分

- 例: $f(x_0, x_1) = x_0^2 \times x_1$ ($x_0 = 10, x_1 = 100$)
- 第一步: 前向计算值

Tape

x_0	x_1	t	y
-------	-------	-----	-----

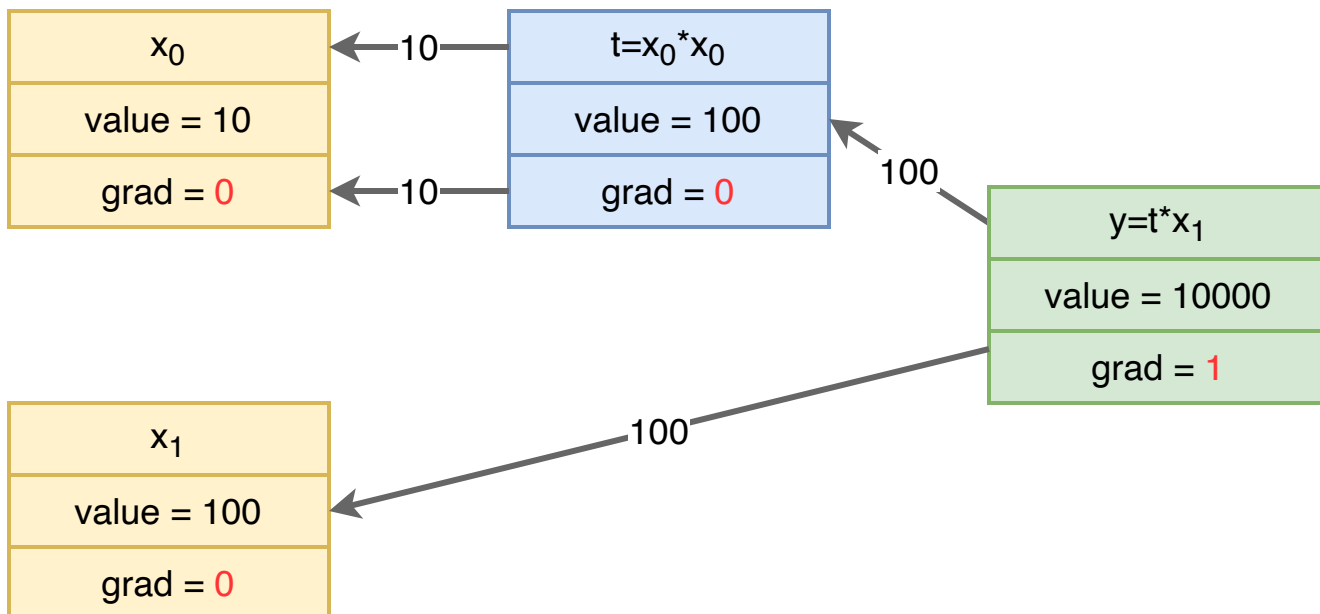


后向微分

- 例: $f(x_0, x_1) = x_0^2 \times x_1$ ($x_0 = 10, x_1 = 100$)
- 第二步: 设输出的导数为1, 其它导数清零

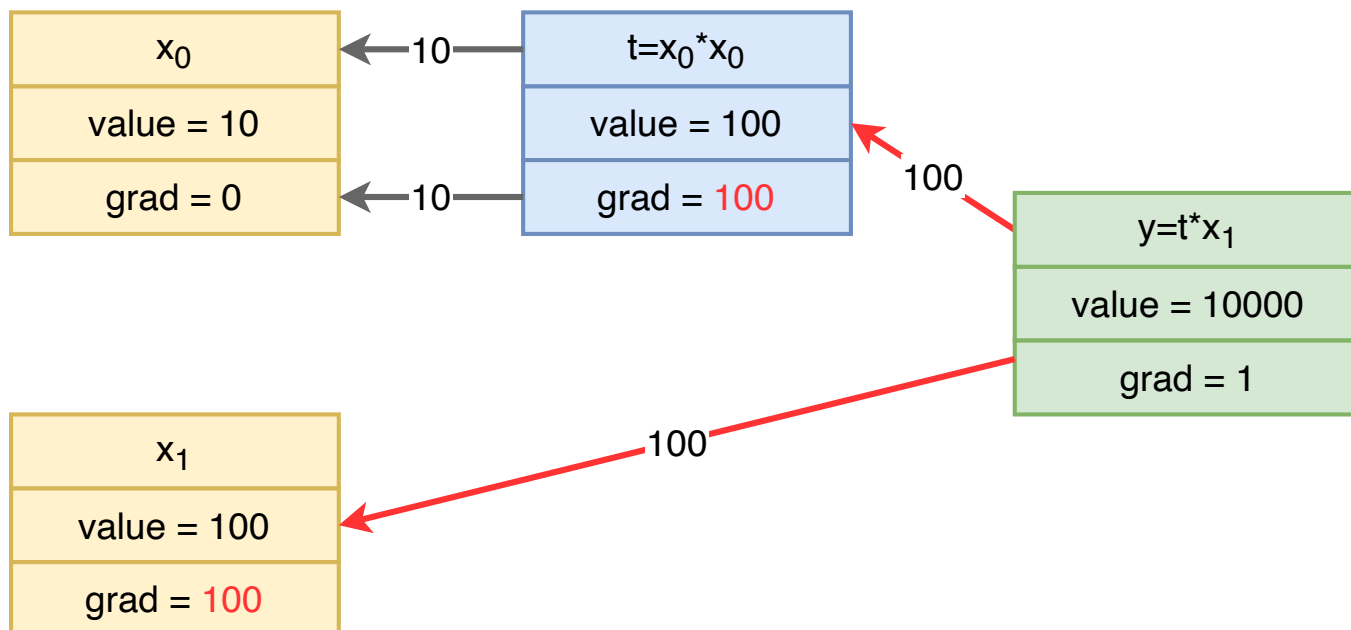
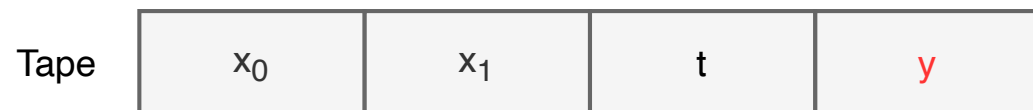
Tape

x_0	x_1	t	y
-------	-------	-----	-----



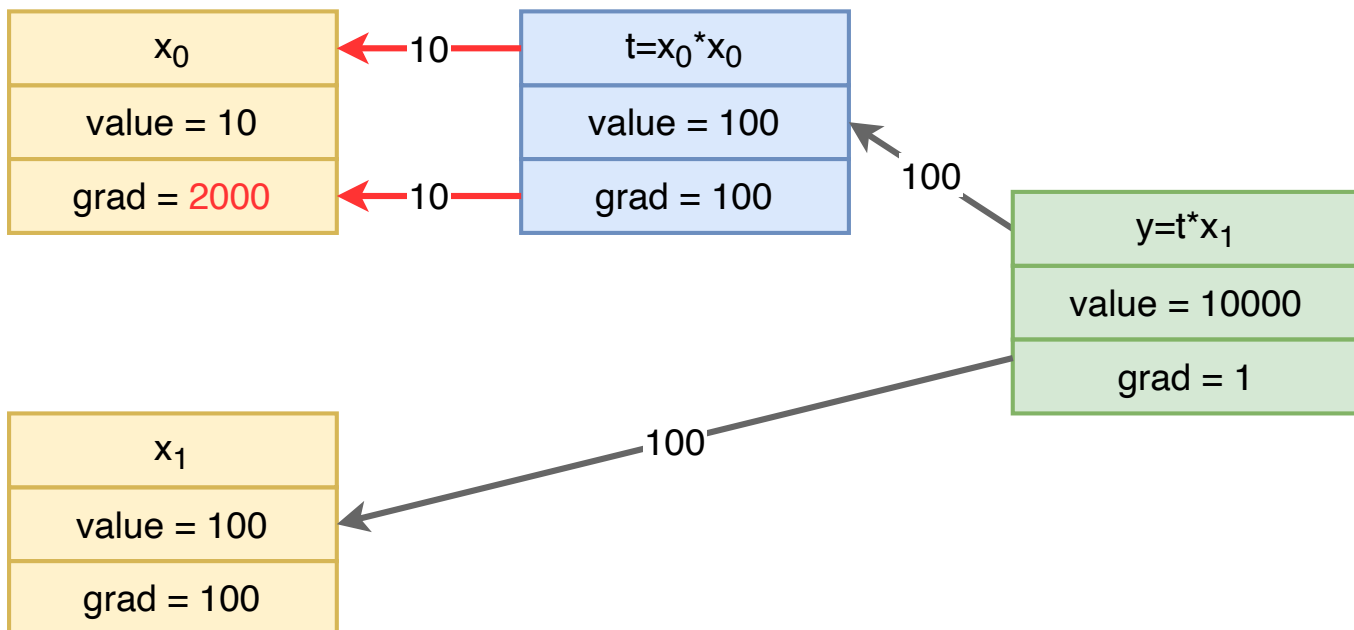
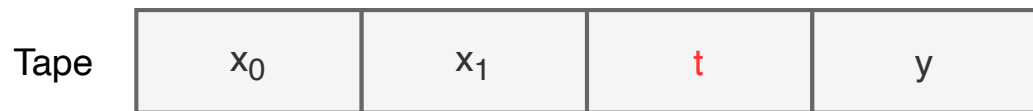
后向微分

- 例: $f(x_0, x_1) = x_0^2 \times x_1$ ($x_0 = 10, x_1 = 100$)
- 第三步: 从后向前计算导数



后向微分

- 例: $f(x_0, x_1) = x_0^2 \times x_1$ ($x_0 = 10, x_1 = 100$)
- 第三步: 从后向前计算导数



后向微分案例：牛顿迭代法求零点

- $f = x^3 - 10x^2 + x + 1$

```
1. fn example_newton_bw(t : Tape, x : Var) -> Var {  
2.   let t1 = t.mul(t.mul(x, x), x)  
3.   let t2 = t.mul(t.input(-10.0), t.mul(x, x))  
4.   t.add(t.add(t.add(t1, t2), x), t.input(1.0))  
5. }
```

后向微分案例：牛顿迭代法求零点

- 利用循环进行迭代： $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$

```
1. test "Newton's method 牛顿迭代法案例" {
2.   let mut x = 1.0
3.   while true {
4.     let tape = Tape::new()
5.     let x_var = tape.input(x)
6.     let y = example_newton_bw(tape, x_var)
7.     tape.backward(y)
8.     if (tape.val(y) / tape.grad(x_var)).abs() < 1.0e-9 {
9.       break
10.    }
11.    x -= tape.val(y) / tape.grad(x_var) // update x 更新 x
12.  }
13.  inspect(x, content=(0.37851665401644224))
14. }
```

总结

- 本章节介绍了自动微分的概念
 - 展示了符号微分
 - 展示了前向微分与后向微分
- 拓展阅读
 - 3Blue1Brown: 深度学习系列 (梯度下降法、后向传播算法)